State Transition Diagram Validation Rules

Anonymous
Anonymous

To assess the structural correctness of UML state transition diagrams, we define validation rules based on UML semantics, implemented in our structural validation framework. Each rule is categorized by its support level in standard UML references, including the *PlantUML Language Reference Guide* [3], *UML Distilled* [2], and *UML 2 and the Unified Process* [1]. Rules are classified as directly supported, conceptually supported, or project heuristics based on practical constraints. These rules combine formal UML semantics with practical heuristics for automated evaluation. While some constraints (e.g., final states, guarded transitions) are defined in UML standards, others (e.g., unreachable or orphan states) are not formally specified but indicate structurally invalid diagrams. This approach provides a comprehensive evaluation of diagrams, bridging theoretical correctness and practical usability.

Validation Rule	Definition	Source Support	Classification
Exactly one initial state transition	Single initial pseudostate with one outgoing transition (e.g., `[*] -> State`)	UML sources conceptually support a single entry point but do not explicitly enforce uniqueness [3, 2, 1]	Project heuristic
At least one final state transition	At least one transition to a final state (e.g., `State -> [*]`)	Explicitly supported in PlantUML and UML lifecycle modeling [3, 2]	Directly supported
No duplicate transitions	No repeated transitions with identical source, destination, and label	Ambiguous or redundant transitions are discouraged in UML semantics [1]	Partly supported + heuristic
No unreachable states	All states must be reachable from the initial state	Not explicitly defined in standard UML references	Project heuristic
No orphan explicit states	States must have at least one incoming or outgoing transition	Not explicitly defined in standard UML references	Project heuristic
Choice node must have outgoing transitions	`«choice»` nodes must define branching behavior	Choice pseudostates are defined as branching constructs in UML [3, 1]	Conceptually supported
Choice node transitions must be guarded	Outgoing transitions from `«choice»` must include guards (e.g., `[cond]`)	Guarded transitions are explicitly supported for alternative execution paths [2, 1]	Directly supported
Fork node must have multiple outgoing branches	`«fork»` splits execution into parallel branches	Fork semantics explicitly represent parallel branching [3]	Directly supported
Join node must have multiple incoming branches	`«join»` merges parallel branches	Join semantics explicitly represent synchronization behavior [3, 1]	Directly supported
History state with composite state	`[H]` and `[H*]` must appear within composite-state contexts	History pseudostates are explicitly defined for composite-state resumption [3, 2, 1]	Directly supported

Table 1 Validation rules used in the static analysis framework and their alignment with UML and PlantUML references.

Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

References

1 Jim Arlow and Ila Neustadt. *UML 2 and the unified process: practical object-oriented analysis and design*. Pearson Education, 2005.

2 Martin Fowler. *UML distilled: a brief guide to the standard object modeling language*. Addison-Wesley Professional, 3rd edition, 2003.

3 PlantUML Team. Plantuml language reference guide. Accessed: 2026-05-06. URL: `https://plantuml.com/guide`.